

Project set-up

1. We will create a Django project called *my_api*, with the following command:

```
django-admin startproject my_api
```

2. Switch to the *my_api* directory and create an app called *books*:

```
./manage.py startapp books
```

3. After these steps, our working directory should look like what's shown below:

```

├── books
│   ├── admin.py
│   ├── apps.py
│   ├── __init__.py
│   ├── migrations
│   │   └── __init__.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── manage.py
└── my_api
    ├── __init__.py
    ├── __init__.pyc
    ├── settings.py
    ├── settings.pyc
    ├── urls.py
    └── wsgi.py

```

Let's start working on the code now.

Creating models

Let's create two models for the *books* app, as follows, in *books/models.py*:

```

class Publication(models.Model):
    """Model for Publication"""
    name = models.CharField(max_length=20)
    isbn = models.CharField(max_length=30)
    website = models.URLField()
    def __unicode__(self):
        return self.name

class Book(models.Model):
    """Model for Book"""
    name = models.CharField(max_length=30, help_text="Name of the Book")
    author = models.CharField(max_length=30)
    description = models.TextField()
    price = models.IntegerField()

```

```

publication = models.ForeignKey(Publication)
def __unicode__(self):
    return self.name

```

Configuration of Tastypie

We only need to add *tastypie* in *INSTALLED_APPS* in *settings.py*.

```

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'books',
    'tastypie',
]

```

Creating resources

For implementing REST APIs, 'resources' needs to be created, and to integrate Tastypie we need to create resource classes. We will create resource classes for each model. So, create the *books/api.py* file, and add resource classes within that file.

```

from tastypie.resources import ModelResource
from books.models import Book, Publication

```

```

class PublicationResource(ModelResource):
    """Resource for Publication"""
    class Meta:
        queryset = Publication.objects.all()
        resource_name = 'publications'

```

```

class BookResource(ModelResource):
    """Resource for Book"""
    class Meta:
        queryset = Book.objects.all()
        resource_name = 'books'

```

As *BookResource* and *PublicationResource* are subclasses of *ModelResource*, it will take all non-relational fields of models and maps to its own *ApiFields*, much like Django *ModelForms*.

In the above class, the *resource_name* is optional. If not provided, it will automatically be generated from the class name, removing any instance of the 'Resource' and lower-casing the string. So 'BookResource' will become only 'book'.